

Uniject / Quick Start

Dependency injection for modular Unity game systems.

1. Install the package

Install Uniject through Unity's Package Manager. For a local UPM copy, use the option to install a package from disk and select Uniject's **package.json**.

2. Set up your scene

Create a GameObject and add **SceneContext**. Save the example below as **GameInstaller.cs**, then add **GameInstaller** to the same GameObject.

3. Register and inject a service

```
using Uniject;
using Uniject.Installers;
using Uniject.Lifecycle;
using UnityEngine;

public sealed class GameInstaller : MonoInstaller
{
    public override void Install(Container container)
    {
        // Share one service instance across consumers.
        container.Bind<ScoreService>().AsCached();
        // Create the entry point and run it at startup.
        container.Bind<GameStartup>().AsEntryPoint();
    }
}

public sealed class ScoreService
{
    public int Value { get; private set; }
    public void Add(int points) => Value += points;
}

public sealed class GameStartup : IEntryPoint
{
    private readonly ScoreService _score;
    // Uniject supplies this constructor dependency.
    public GameStartup(ScoreService score) => _score = score;

    public void Run()
    {
        _score.Add(10);
        Debug.Log(_score.Value); // Prints 10.
    }
}
```

Press Play. Uniject injects **ScoreService** into **GameStartup** and calls **Run**. The Console displays **10**.

Continue with the full documentation

[Open the Uniject documentation in Notion](#)

Explore bindings, MonoBehaviour injection, subcontainers, factories, pools, signals, and object lifecycles. Internet access is required for Notion.